

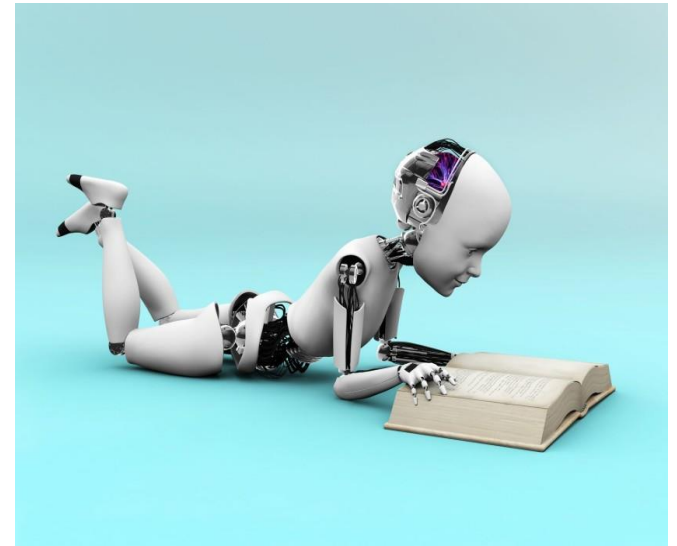


Artificial Neural Networks

Dr. Huseyin Kusetogullari

Learning Outcomes

- Motivation
- Why do we need ANN?
- Application areas of ANNs
- What is NN?
- Activation functions used in ANNs
- Gradient Descents
- Perceptron
 - Learning Algorithm
 - Numerical Example
- Advanced Learning Methods
 - Multi-layer Neural Networks
 - Recurrent Neural Networks



Intelligence



The Brain's Role in Processing and Decision-Making

Emotions and Social Intelligence

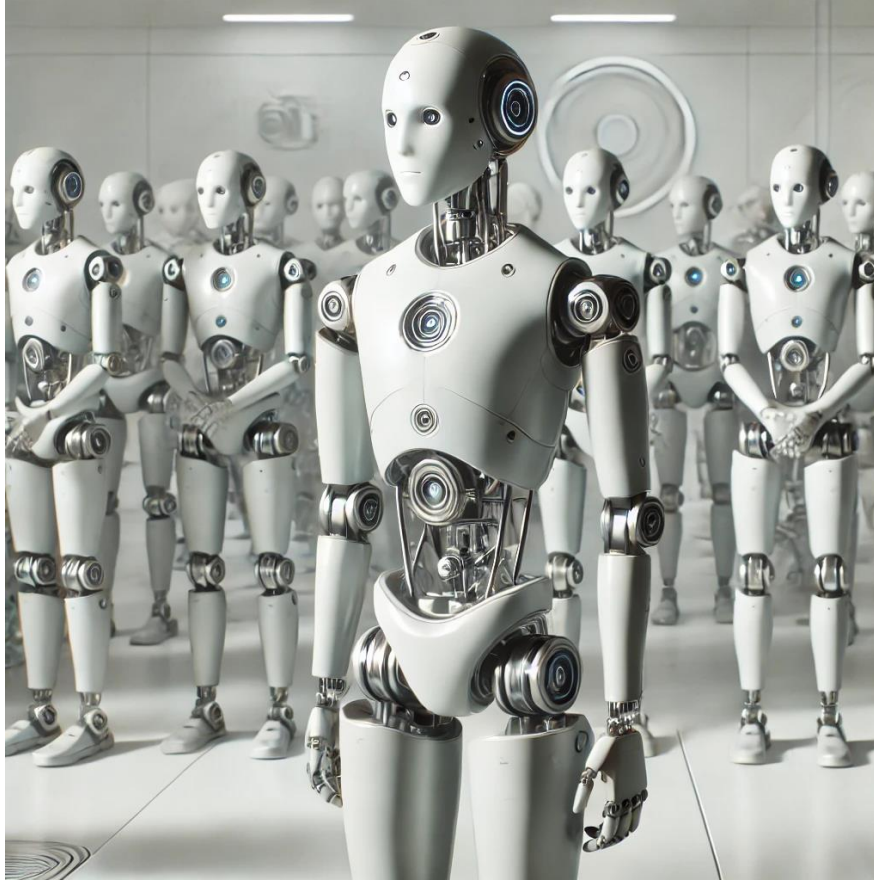
Learning and Memory

Creativity and Problem-Solving



The human brain contains approximately **120 billion neurons [1]**. These neurons are the brain's primary cells, responsible for transmitting information through electrical and chemical signals.

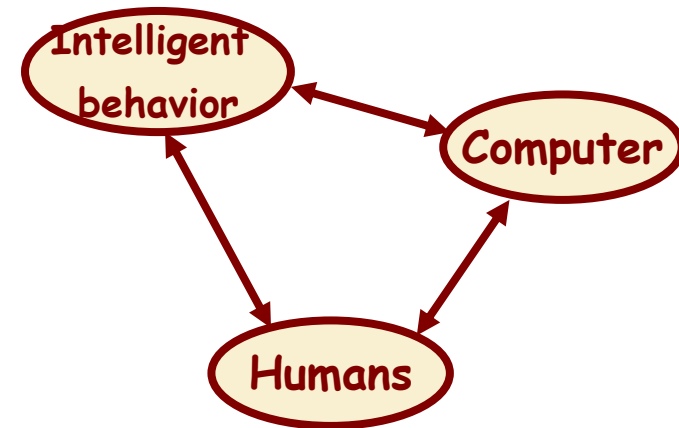
What is Intelligence?

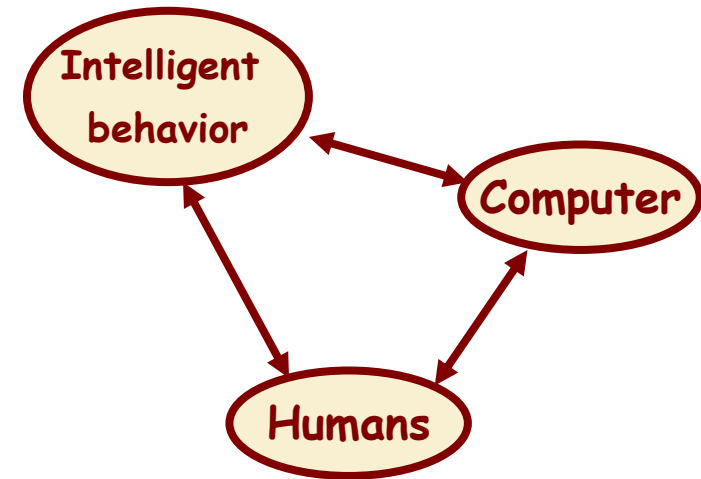


Basic Robots or Machines: no decision-making ability or self-awareness

What is Artificial Intelligence?

- is a simulation of human intelligence processes by machines, especially computer systems.
- AI refers to the field of computer science dedicated to creating systems or machines that can perform tasks typically requiring human intelligence.
- is defined as the ability of a digital computer or computer-controlled robot to perform tasks commonly associated with human intelligent beings.
- is the intelligence of a machine or computer that enables it to imitate or mimic human capabilities.
- Creating computer systems that perform tasks regarded as requiring intelligence when done by humans.





Motivation

- An Artificial Intelligence method makes human life easier, safer and better.
- An Artificial Intelligence method is useful for too complex tasks
- For humans, these complex tasks can be
 - Impractical
 - Time consuming
 - Hard to do or solve



Motivation

- Data is everywhere in abundant amounts.
- With humans effort, it is hard to extract useful information from these data sets.
- An automatic intelligence method may explore the data to predict the result.
- For instance:
 - Surveillance cameras continuously capture video,
 - Satellites generate tera-bytes of data every year,
 - Communication companies want to understand clients' requests,
 - Soon all written texts from our libraries will be digitized etc.
- Lets assume that a car stolen in a capital city. To find it, it is necessary to search the licence plate of the car from the 24 hours working 50 different surveillance cameras.
 - Can human find the car by manual search?



Application Areas

- Facebook
- Youtube: Videos
- Instagram: Images
- Twitter: words, sentences etc.

Year	1	2	3	4	5	6	7	8	9	10
2000	pro	mahjong	series	racing	star	world	dance	de	1500	simple
2001	series	simple	pro	x	world	1500	advance	racing	super	hunter
2002	pro	series	advance	simple	world	Wars	soccer	star	nba	tennis
2003	series	world	football	battle	pro	club	racing	star	game	dragon
2004	football	club	series	man	world	battle	famicom	mini	classic	final
2005	world	man	Wars	racing	series	collection	king	star	battle	x
2006	world	battle	ds	racing	series	dance	fifa	game	portable	ultimate
2007	ds	game	world	battle	series	monster	quest	dance	war	man
2008	world	ds	game	Wars	party	american	episode	star	war	dance
2009	episode	world	Wars	game	pocket	god	hero	star	force	vs
2010	world	game	super	episode	pro	dead	star	Wars	alien	heroes
2011	3d	game	war	world	adventures	space	super	dead	man	vs
2012	3d	game	hd	legends	super	war	edition	heroes	black	dark
2013	hd	world	dead	dungeon	space	star	war	Wars	super	3d
2014	war	dead	putt	world	space	adventure	dark	planet	time	game
2015	life	shadows	war	dark	elysian	fate	night	battlefield	hand	hardline

Categorical Data: words

	A	B	C	D	E	F	G	H	I	J	K	L
1	fixed acid	volatile ac	citric acid	residual si	chlorides	free sulfu	total sulfu	density	pH	sulphates	alcohol	quality
2	7.4	0.7	0	1.9	0.076	11	34	0.9978	3.51	0.56	9.4	5
3	7.8	0.88	0	2.6	0.098	25	67	0.9968	3.2	0.68	9.8	5
4	7.8	0.76	0.04	2.3	0.092	15	54	0.997	3.26	0.65	9.8	5
5	11.2	0.28	0.56	1.9	0.075	17	60	0.998	3.16	0.58	9.8	6
6	7.4	0.7	0	1.9	0.076	11	34	0.9978	3.51	0.56	9.4	5
7	7.4	0.66	0	1.8	0.075	13	40	0.9978	3.51	0.56	9.4	5
8	7.9	0.6	0.06	1.6	0.069	15	59	0.9964	3.3	0.46	9.4	5
9	7.3	0.65	0	1.2	0.065	15	21	0.9946	3.39	0.47	10	7
10	7.8	0.58	0.02	2	0.073	9	18	0.9968	3.36	0.57	9.5	7
11	7.5	0.5	0.36	6.1	0.071	17	102	0.9978	3.35	0.8	10.5	5
12	6.7	0.58	0.08	1.8	0.097	15	65	0.9959	3.28	0.54	9.2	5
13	7.5	0.5	0.36	6.1	0.071	17	102	0.9978	3.35	0.8	10.5	5
14	5.6	0.615	0	1.6	0.089	16	59	0.9943	3.58	0.52	9.9	5
15	7.8	0.61	0.29	1.6	0.114	9	29	0.9974	3.26	1.56	9.1	5
16	8.9	0.62	0.18	3.8	0.176	52	145	0.9986	3.16	0.88	9.2	5
17	8.9	0.62	0.19	3.9	0.17	51	148	0.9986	3.17	0.93	9.2	5
18	8.5	0.28	0.56	1.8	0.092	35	103	0.9969	3.3	0.75	10.5	7
19	8.1	0.56	0.28	1.7	0.368	16	56	0.9968	3.11	1.28	9.3	5
20	7.4	0.59	0.08	4.4	0.086	6	29	0.9974	3.38	0.5	9	4
21	7.9	0.32	0.51	1.8	0.341	17	56	0.9969	3.04	1.08	9.2	6
22	8.9	0.22	0.48	1.8	0.077	29	60	0.9968	3.39	0.53	9.4	6
23	7.6	0.28	0.21	2.2	0.092	22	71	0.9967	2.67	0.65	9.7	6

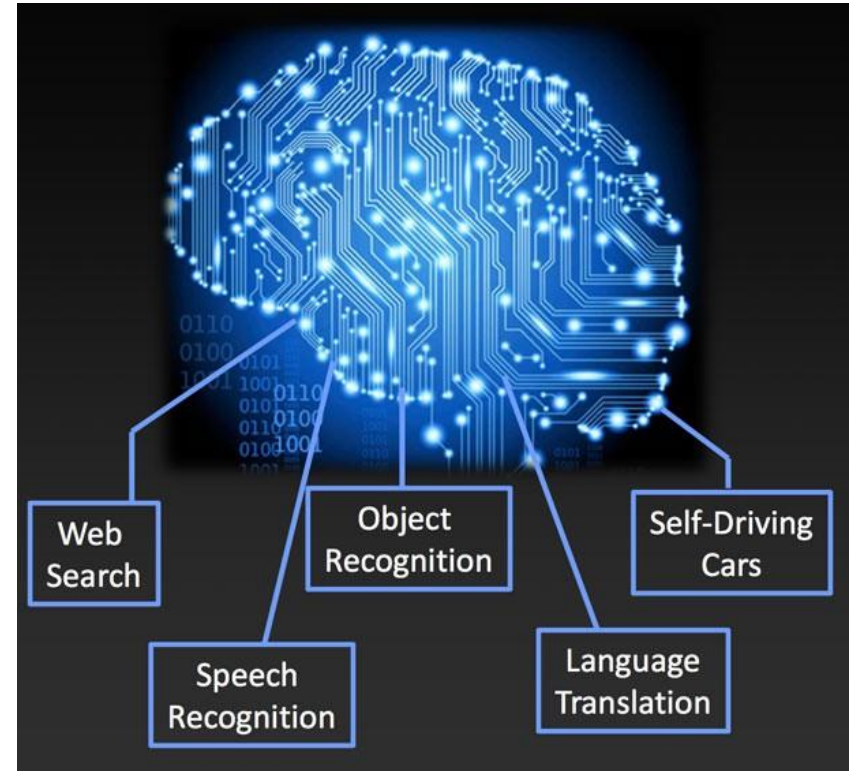
Numerical Data



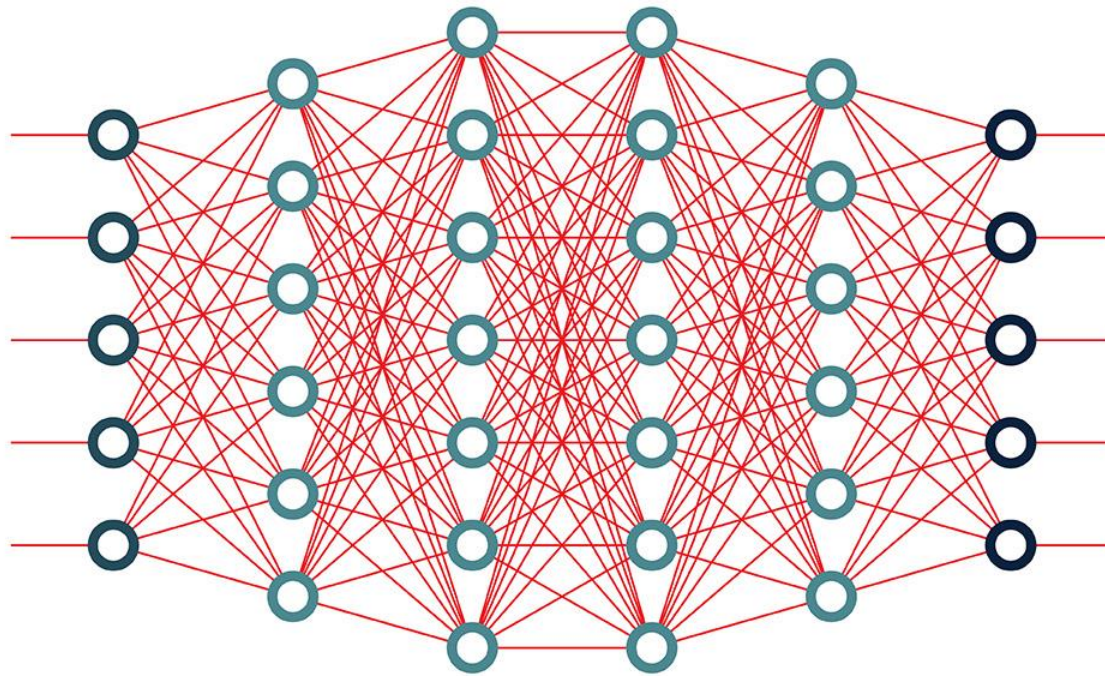
Visual data: images and Videos

Cont.

- Object Recognition images
- Biometric Recognition
- Web search
- Wind power prediction
- Speech Recognition
- Language Translation

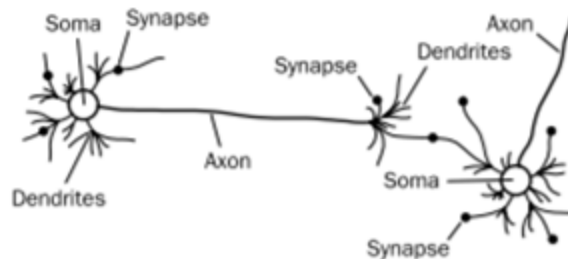


An Artificially Designed Neural Network Model



What is Neural Network?

- A neural network can be defined as a model of reasoning based on the human brain (the average human brain has about 100 billion neurons (or nerve cells)).
- The brain consists of interconnected set of nerve cells, or basic information-processing units, called **neurons**.
- Signals are propagated from one neuron to another by complex electrochemical reactions.



Biological neural network

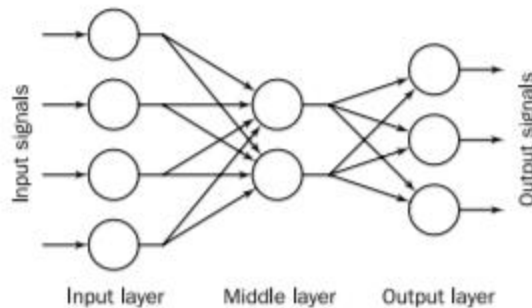
Biological neural network	Artificial neural network
Soma	Neuron
Dendrite	Input
Axon	Output
Synapse	Weight

What is Neural Networks?

- Our brain can be considered as a highly complex, nonlinear and parallel information-processing system.
- Information is stored and processed in a neural network simultaneously throughout the whole network, rather than at specific locations.
- ANNs are capable of 'learning', that is, they use experience to improve their performance.

What is Neural Network?

- The neurons are connected by weighted links passing signals from one neuron to another.
- Each neuron receives a number of input signals through its connections; however, it **never** produces more than a single **output signal**.



Architecture of a typical artificial neural network

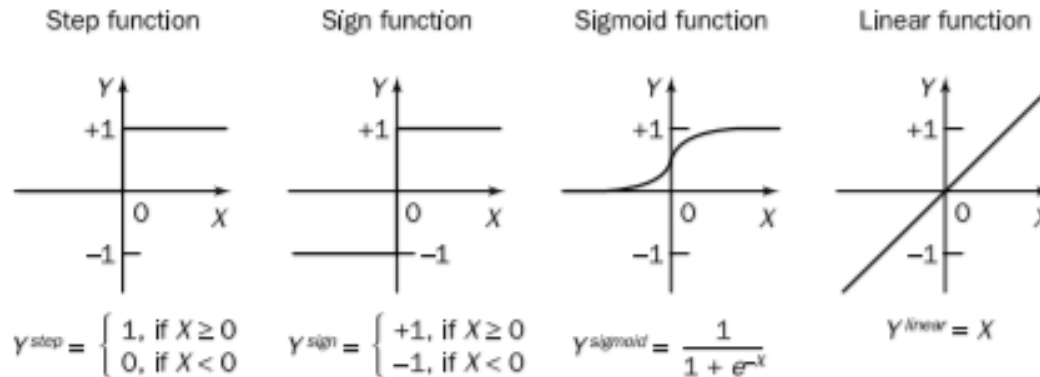
Background

- Supervised based
- Train the Artificial Neural Network model
- Need labelled data to train the model
- Need test data to understand the performance of the trained ANN
- Iteratively train the model

Activation Functions

- Many activation functions have been proposed, but only a few have found practical applications.
- Their main purpose is to convert a input signal of a node in a A-NN to an output signal.
- Activation functions are important to avoid using large values in NN (the unbounded linearly weighted sum from neurons).
- Help the network to use the useful information (decrease the noise effect).
- Four common choices – the step, sign, sigmoid and linear.
- The step and sign activation functions, also called hard limit functions.
- Linear and sigmoid activation functions are soft limit functions.

Activation Functions



Activation functions of a neuron

- Example: Use find y (output) for x=5 (input) and use four activation functions
 - y = +1 (Step function)
 - y = +1 (Sign function)
 - y = $1/1+e^{-5} = 0.9933$ (Sigmoid function)
 - y = 5 (Linear function)

Activation Functions (Contd.)

- **Tanh (tanch):**

- A variant of the Sigmoid function varies from 0 to 1
- Caters for scenarios where we would like to **introduce negative sign** to the output of artificial neuron. This is where hyperbolic tangent function becomes useful
- Still suffers the phenomena of vanishing gradient

- **ReLU (Rectified Linear Unit):**

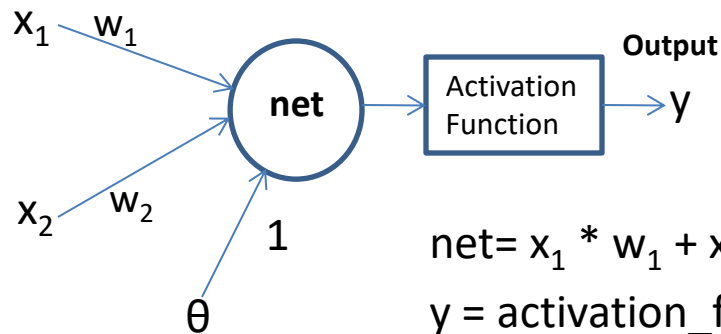
- Nonlinear and Differentiable
- Does not suffer vanishing gradient $\max(0, w \cdot x + b)$; that the neural network will **NOT** learn slowly and poorly.
- Dies out on negative derivatives
- ReLU is computationally fast and shows great results.

<https://machinelearningknowledge.ai/activation-functions-neural-network/>
<https://mlfromscratch.com/activation-functions-explained/>
<https://aideepdive.com/hyper-parameters-in-action-activation-functions/>

Perceptron

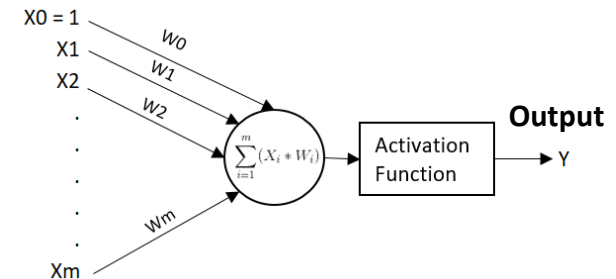
- Is the simplest form of a neural network,
- Perceptron is a linear classifier,
- Consists of a single neuron with adjustable weights and bias,
- A neuron receives several signals from its input links, computes a new activation level and sends it, as an output signal through the output link(s),
- The input signal can be raw data or outputs of other neurons,
- The output signal can be either a final solution to the problem or an input to other neurons.

Inputs

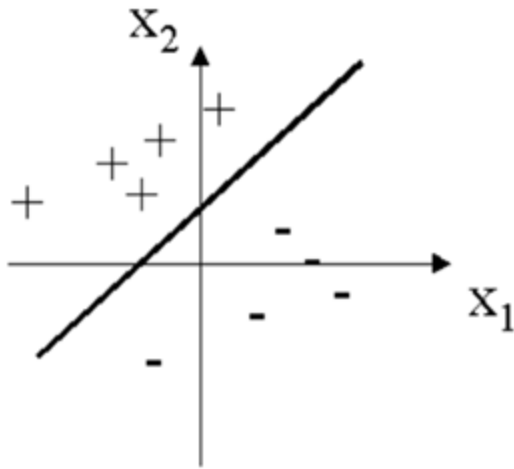


$$\text{net} = x_1 * w_1 + x_2 * w_2 + \theta * 1$$
$$y = \text{activation_function}(\text{net})$$

Inputs

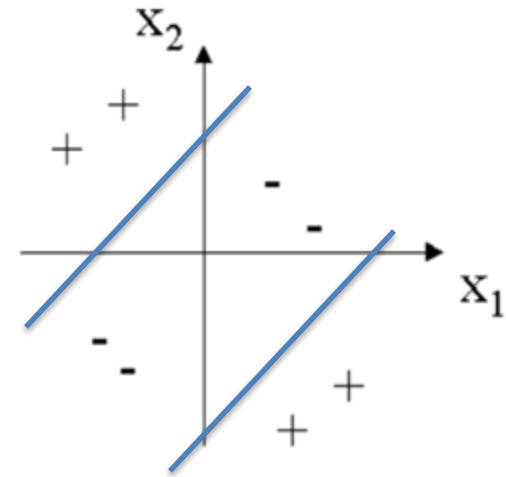
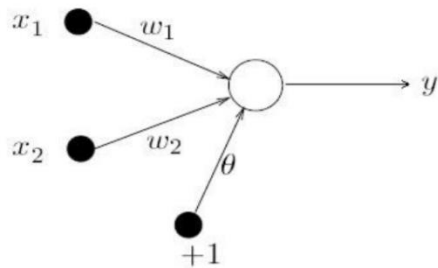


Perceptron



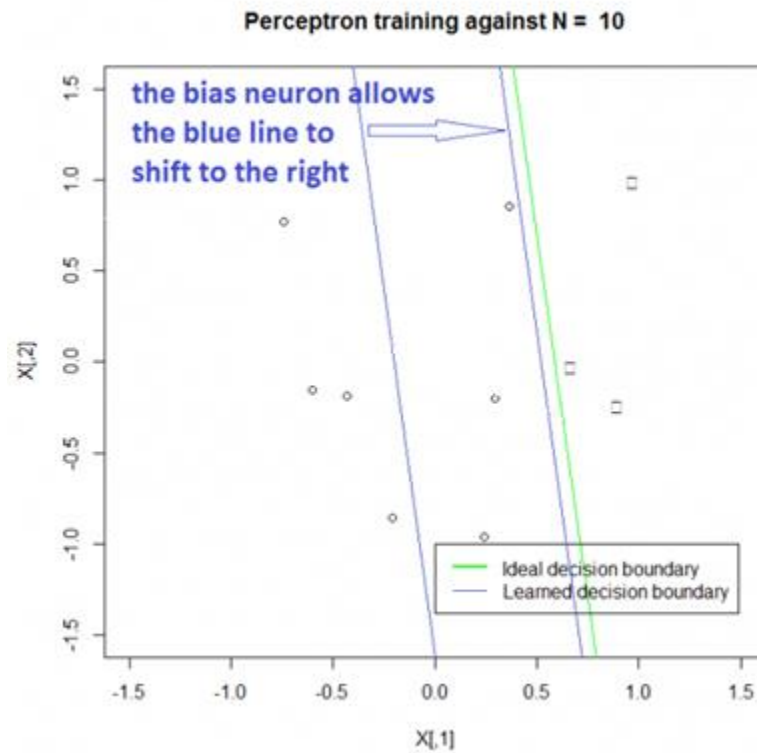
Classification with one Perceptron

$$w_1x_1 + w_2x_2 + \theta = 0$$



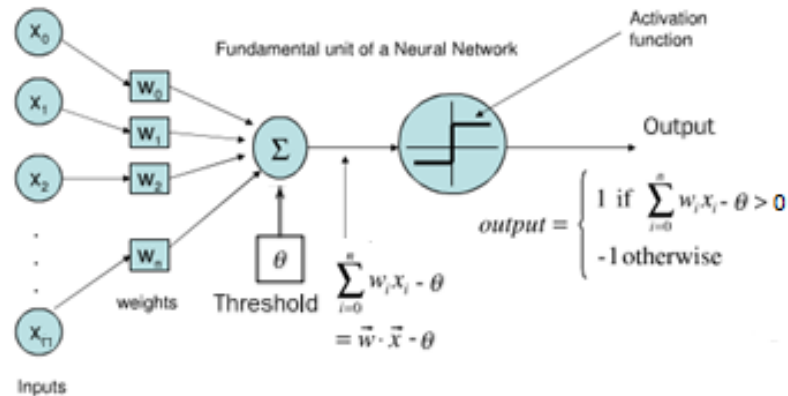
Classification with more than one Perceptron

Perceptron



How does a perceptron learn its classification tasks?

- By updating weights
- By reducing the difference between the actual and desired outputs of the perceptron.
- Iteration based



Gradient Descents

- gradient descent is an algorithm that minimizes functions.
- iteratively moves toward a set of parameter values that minimize the function.
- This iterative minimization is achieved using calculus, taking steps in the negative direction of the function gradient.
- The optimization formula is given as follows:

$$\theta = \theta - \text{learning_rate} * \frac{\partial f}{\partial \theta}$$

Example: Gradient Descents

- Minimize $f(x) = x^2$ using Gradient Descent method.

- Answer:

Derivative of $f(x)$: $\frac{\partial f}{\partial x} = f'(x) = 2x$

Assume that: learning_rate=0.1, initial_x=11

Example: Gradient Descents

$$x = x - \text{learning_rate} * 2 * x$$

$$x=11, f(x^2)= 121$$

First iteration:

$$x=11 - 0.1 * 2 * 11 = 8.8$$

$$f(x^2)=77.44$$

Iteration 2:

$$x = 8.8 - 0.1 * 2 * 8.8 = 7.04$$

$$f(x^2)= 49.56$$

Iteration 3:

$$x = 7.04 - 0.1 * 2 * 7.04 = 5.63$$

$$f(x^2)= 31.71$$

Iteration 3:

$$x = 5.63 - 0.1 * 2 * 5.63 = 4.50$$

$$f(x^2)= 20.30$$

...

Iteration 50:

$$x = 0.000245 - 0.1 * 2 * 0.000245 = 0.000196$$

$$f(x^2)= 0.0000000385$$

Code in Python:

```
import numpy as np
import matplotlib.pyplot as plt

lr=0.1
x=11

X=[]
Y=[]
for i in range(50): #Iteration number is 50
    X.append(x) #store the result value in X
    y=x**2
    Y.append(y)
    print(x, 'x^2=', y) #Display the optimized result
    x = x - lr*2*x #find the optimal result using gradient descent
```

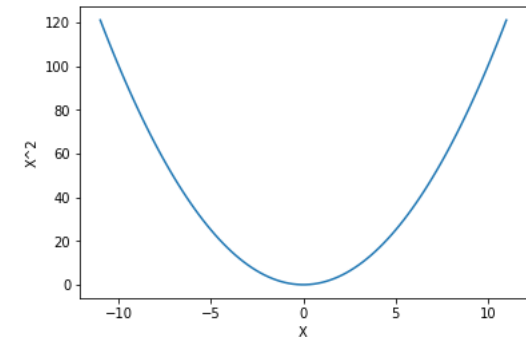
```
#Create the data between -11 and +11 to draw the x^2 line
x = np.linspace(-11, 11, 100)
y = x**2
```

```
fig, ax = plt.subplots()
plt.ylabel('X^2')
plt.xlabel('X')
ax.plot(x, y)
plt.show()
```

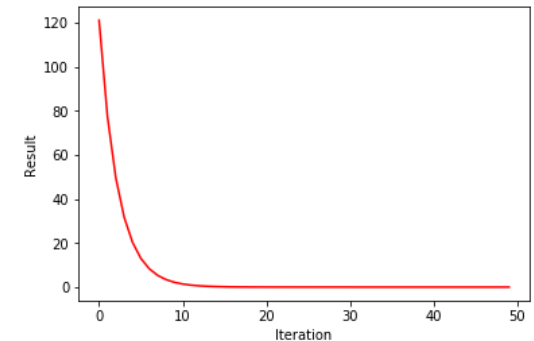
```
fig, ax = plt.subplots()
plt.ylabel('Result')
plt.xlabel('Iteration')
ax.plot(Y, 'r-')
plt.show()
```

```
fig, ax = plt.subplots()
ax.plot(x, y, X, 'r-')
plt.show()
```

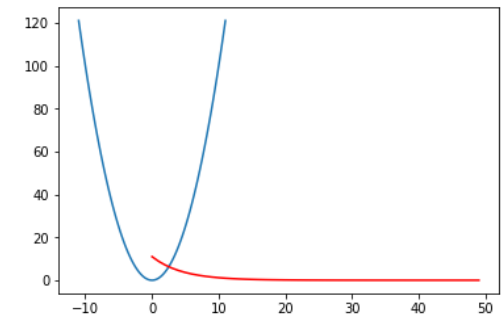
X^2 graph



Optimization of X^2 graph



Combination Of two graph



Types of Gradient Descents

- There are three well known gradient descent methods applied to the ANNs and Deep Learning methods:
 - **Classical Gradient Descent:** Train *whole* dataset and find the error then apply the gradient descent to the ANN to update the weights. It takes long time to update the model
 - **Stochastic Gradient Descent:** Train *each* data point and find the error then apply the gradient descent to the ANN to update the weights. This approach is much faster than Classical Gradient Descent.
 - **Mini-Batch Gradient Descent:** Train *some* data in the dataset and find the error then apply the gradient descent to the ANN to update the weights. This approach achieves the optimal weight result fast and efficiently.

Perceptron Learning Algorithm

- Initialize **weights** and **bias** values randomly
- Compute the output from the perceptron using

$$\text{net}(p) = x_1(p) * w_1(p) + x_2(p) * w_2(p) + \dots + \theta(p) * 1 = \sum_{i=0}^N W_i x_i + \theta$$

$$y(p) = \text{activation_function}(\text{net}(p))$$

- Update the weight values by

$$w_i(p+1) = w_i(p) + \eta * x_i(p) * e(p)$$

$$\text{and error } e(p) \text{ is defined as } e(p) = y(d) - y(p)$$

- If output is correct then do nothing
- If output is high, then decrease weight values
- If output is low, then increase weight values

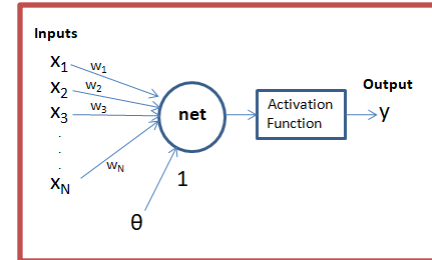
- Update the **bias** by

$$\theta(p+1) = \theta(p) + \eta * \theta(p) * e(p)$$

- Minimize the total error

$$E_{total} = \frac{1}{2} \sum_{i=1}^n (y_d^i - y^i)^2$$

- Increase iteration number



Learning rate

Computed output

Desired output (labelled)

Training basic logical operations: AND, OR or Exclusive-OR

- The table presents all possible combinations of values for two variables, x_1 and x_2 , and the results of the operations.
- The perceptron must be trained to classify the input patterns

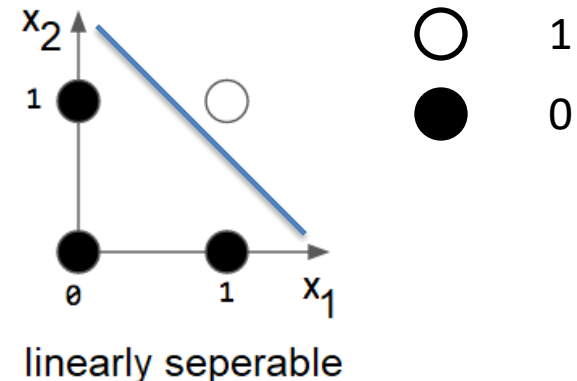
Truth tables for the basic logical operations

Input variables		AND	OR	Exclusive-OR
x_1	x_2	$x_1 \cap x_2$	$x_1 \cup x_2$	$x_1 \oplus x_2$
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Example

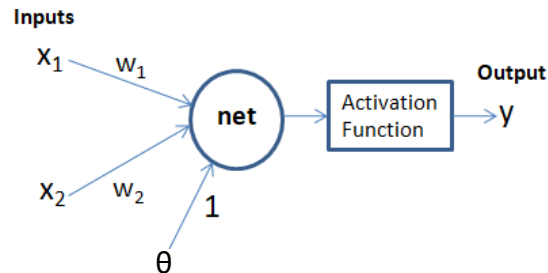
- Let us first consider the operation AND.
- After completing the initialisation step, the perceptron is activated by the sequence of four input patterns representing an epoch.
- The perceptron weights are updated after each activation. This process is repeated until all the weights converge to a uniform set of values

Input variables		AND
x_1	x_2	$x_1 \cap x_2$
0	0	0
0	1	0
1	0	0
1	1	1



Example

Input variables		AND
x_1	x_2	$x_1 \cap x_2$
0	0	0
0	1	0
1	0	0
1	1	1



Initial values: $w_1=0.3$, $w_2=-0.1$, $\theta=-0.2$
 Learning rate=0.1

$$x_1=0, x_2=0$$

- $y(p) = \text{step}(0 \times 0.3 + 0 \times (-0.1) - 0.2) = \text{step}(-0.2) = 0$

$$e(p) = y(d) - y = 0 - 0 = 0$$

$$w_1 = 0.3 + 0.1 \times 0 \times 0 = 0.3$$

$$w_2 = -0.1 + 0.1 \times 0 \times 0 = -0.1$$

$$\theta = -0.2 + 0.1 \times -0.2 \times 0 = -0.2$$

$$Y(p) = \text{step} \left[\sum_{i=1}^n x_i(p) w_i(p) + \theta \right]$$

$$e(p) = y(d) - y(p)$$

Update the weight and bias of the perceptron

$$w_i(p+1) = w_i(p) + \eta * x_i(p) * e(p)$$

$$\theta(p+1) = \theta(p) + \eta * \theta(p) * e(p)$$



Example with one Perceptron:

w1=0.3, w2=-0.1, θ=-0.2

Learning rate=0.1

- $y = \text{step}(0 \times 0.3 + 1 \times (-0.1) - 0.2) = \text{step}(-0.3) = 0$

$$x_1=0, x_2=1$$

$$w1 = 0.3 + 0.1 \times 0 \times 0 = 0.3$$

$$w2 = -0.1 + 0.1 \times 1 \times 0 = -0.1$$

$$\theta = -0.2 + 0.1 \times -0.2 \times 0 = -0.2$$

- $y = \text{step}(1 \times 0.3 + 0 \times (-0.1) - 0.2) = \text{step}(0.1) = 1$

$$x_1=1, x_2=0$$

$$w1 = 0.3 + 0.1 \times 1 \times -1 = 0.2$$

$$w2 = -0.1 + 0.1 \times 0 \times -1 = -0.1$$

$$\theta = -0.2 + 0.1 \times -0.2 \times -1 = -0.18$$

- $y = \text{step}(1 \times 0.2 + 1 \times (-0.1) - 0.18) = \text{step}(-0.08) = 0$

$$x_1=1, x_2=1$$

$$w1 = 0.2 + 0.1 \times 1 \times 1 = 0.3$$

$$w2 = -0.1 + 0.1 \times 1 \times 1 = 0$$

$$\theta = -0.18 + 0.1 \times -0.18 \times 1 = -0.198$$

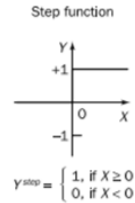
$$Y(p) = \text{step} \left[\sum_{i=1}^n x_i(p) w_i(p) + \theta \right]$$

$$e(p) = y(d) - y(p)$$

Update the weight and bias of the perceptron

$$w_i(p+1) = w_i(p) + \eta * x_i(p) * e(p)$$

$$\theta(p+1) = \theta(p) + \eta * \theta(p) * e(p)$$



Example of perceptron learning for the logical operation AND

	Inputs		Desired Outputs	Weights		Bias	Estimated Output	Error
Iteration	x_1	x_2	Y_d	w_1	w_2	θ	y	e
1	0	0	0	0.3	-0.1	-0.2	0	0
	0	1	0	0.3	-0.1	-0.2	0	0
	1	0	0	0.2	-0.1	-0.18	1	-1
	1	1	1	0.3	0	-0.198	0	1

$$E_{\text{total}} = \frac{1}{2} \sum_{i=1}^n (y_d^i - y^i)^2 = \frac{1}{2} (0 + 0 + 1 + 1) = 1$$

Example with one perceptron:

$$y = \text{step}(0 \times 0.3 + 0 \times (0) - 0.198) = \text{step}(-0.198) = 0$$

$$x_1=0, x_2=0$$

$$w_1 = 0.3 + 0.1 \times 0 \times 0 = 0.3$$

$$w_2 = 0 + 0.1 \times 0 \times 0 = 0$$

$$\theta = -0.198 + 0.1 \times -0.198 \times 0 = -0.198$$

$$y = \text{step}(0 \times 0.3 + 1 \times (0) - 0.198) = \text{step}(-0.198) = 0$$

$$x_1=0, x_2=1$$

$$w_1 = 0.3 + 0.1 \times 0 \times 0 = 0.3$$

$$w_2 = 0 + 0.1 \times 1 \times 0 = 0$$

$$\theta = -0.198 + 0.1 \times -0.198 \times 0 = -0.198$$

$$y = \text{step}(1 \times 0.3 + 0 \times (0) - 0.198) = \text{step}(0.102) = 1$$

$$x_1=1, x_2=0$$

$$w_1 = 0.3 + 0.1 \times 1 \times -1 = 0.2$$

$$w_2 = 0 + 0.1 \times 0 \times -1 = 0$$

$$\theta = -0.198 + 0.1 \times -0.198 \times -1 = -0.1782$$

$$y = \text{step}(1 \times 0.2 + 1 \times (0) - 0) = \text{step}(0.2) = 1$$

$$x_1=1, x_2=1$$

$$w_1 = 0.2 + 0.1 \times 1 \times 0 = 0.2$$

$$w_2 = 0 + 0.1 \times 1 \times 0 = 0$$

$$\theta = -0.1782 + 0.1 \times -0.1782 \times 0 = -0.1782$$

Example of perceptron learning for the logical operation AND

	Inputs		Desired Outputs	Weights		Bias	Estimated Output	Error
Iteration	x_1	x_2	Y_d	w_1	w_2	θ	y	e
2	0	0	0	0.3	0	-0.198	0	0
	0	1	0	0.3	0	-0.198	0	0
	1	0	0	0.2	0	-0.1782	1	-1
	1	1	1	0.2	0	-0.1782	1	0

$$E_{total} = \frac{1}{2} \sum_{i=1}^n (y_d^i - y^i)^2 = \frac{1}{2} (0 + 0 + 1 + 0) = 0.5$$

Example with one perceptron:

$$y = \text{step}(0 \times 0.1 + 0 \times (0.1) - 0.1588) = \text{step}(-0.1588) = 0$$

$$x_1=0, x_2=0$$

$$w_1 = 0.1 + 0.1 \times 0 \times 0 = 0.1$$

$$w_2 = 0.1 + 0.1 \times 0 \times 0 = 0.1$$

$$\theta = -0.1588 + 0.1 \times -0.3 \times 0 = -0.1588$$

$$y = \text{step}(0 \times 0.1 + 1 \times (0.1) - 0.1588) = \text{step}(-0.0588) = 0$$

$$x_1=0, x_2=1$$

$$w_1 = 0.1 + 0.1 \times 0 \times 0 = 0.1$$

$$w_2 = 0.1 + 0.1 \times 1 \times 0 = 0.1$$

$$\theta = -0.1588 + 0.1 \times -0.1588 \times 0 = -0.1588$$

$$y = \text{step}(1 \times 0.1 + 0 \times (0.1) - 0.1588) = \text{step}(-0.0588) = 0$$

$$x_1=1, x_2=0$$

$$w_1 = 0.1 + 0.1 \times 1 \times 0 = 0.1$$

$$w_2 = 0.1 + 0.1 \times 0 \times 0 = 0.1$$

$$\theta = -0.1588 + 0.1 \times -0.1588 \times 0 = -0.1588$$

$$y = \text{step}(1 \times 0.2 + 1 \times (0.2) - 0) = \text{step}(0.1) = 1$$

$$x_1=1, x_2=1$$

$$w_1 = 0.1 + 0.1 \times 1 \times 0 = 0.1$$

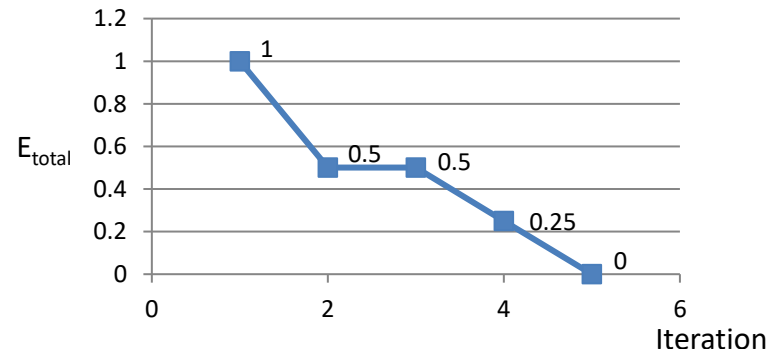
$$w_2 = 0.1 + 0.1 \times 1 \times 0 = 0.1$$

$$\theta = -0.1588 + 0.1 \times -0.158 \times 0 = -0.1588$$

Example of perceptron learning for the logical operation AND

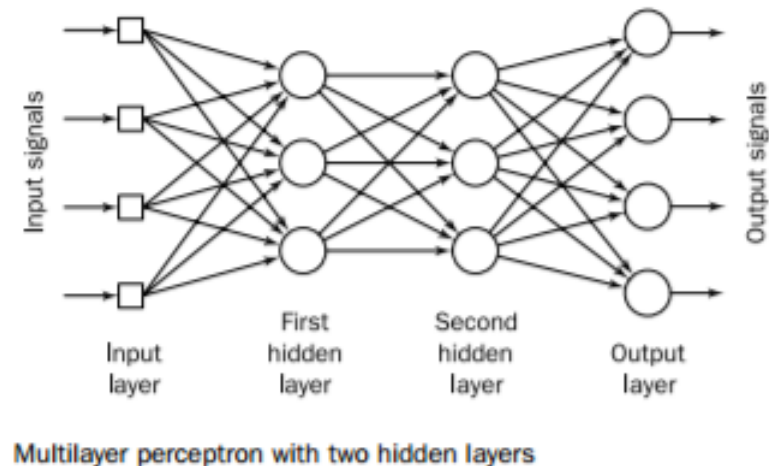
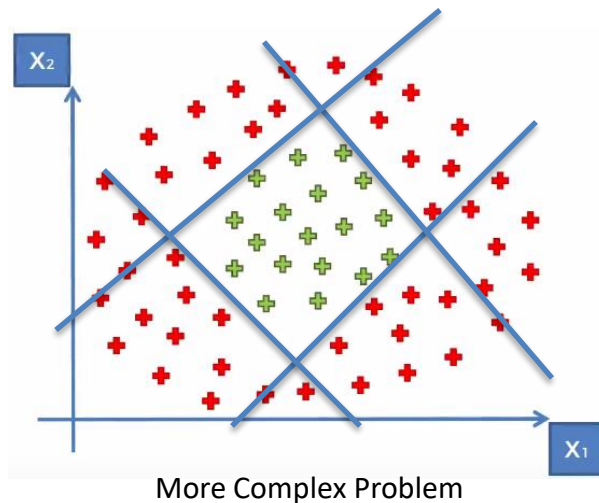
	Inputs		Desired Outputs	Weights		Bias	Estimated Output	Error
Iteration	x_1	x_2	Y_d	w_1	w_2	θ	y	e
5	0	0	0	0.1	0.1	-0.1588	0	0
	0	1	0	0.1	0.1	-0.1588	0	0
	1	0	0	0.1	0.1	-0.1588	0	0
	1	1	1	0.1	0.1	-0.1588	1	0

$$E_{total} = \frac{1}{2} \sum_{i=1}^n (y_d^i - y^i)^2 = \frac{1}{2} (0 + 0 + 0 + 0) = 0$$



Multilayer neural networks

- Multilayer neural networks are more complex neural network models than the single perceptron model.
- These network models consist of number of perceptrons.
- They have input layer, hidden layer(s) and output layer.
- The learning algorithm is based on
 - Feed-forward
 - Back-propagation



Multilayer neural networks

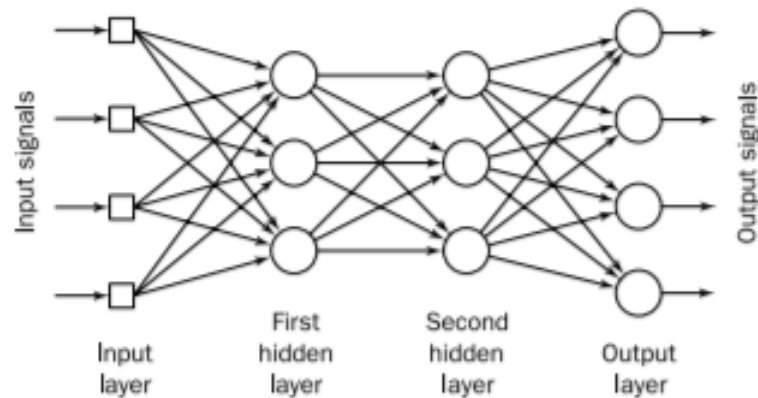
- A multilayer perceptron is a feedforward neural network with one or more hidden layers.
- Typically, the network consists of an input layer of source neurons, at least one middle or hidden layer of computational neurons, and an output layer of computational neurons.
- The input signals are propagated in a forward direction on a layer-by-layer basis.
- A multilayer perceptron with two hidden layers is shown in next slide.

Multilayer neural networks

- Each layer in a multilayer neural network has its own specific function.
- The input layer accepts input signals from the outside world and redistributes these signals to all neurons in the hidden layer.
- Actually, the input layer rarely includes computing neurons, and thus does not process input patterns.
- The output layer accepts output signals, or in other words a stimulus pattern, from the hidden layer and establishes the output pattern of the entire network.

Multilayer neural networks

- Neurons in the hidden layer detect the features; the weights of the neurons represent the features hidden in the input patterns.
- These features are then used by the output layer in determining the output pattern.



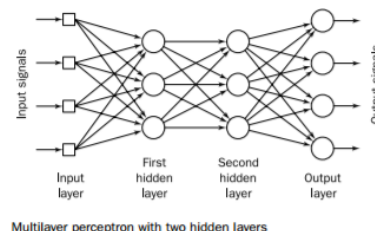
Multilayer perceptron with two hidden layers

Hidden layer

- Why is a middle layer in a multilayer network called a 'hidden' layer? What does this layer hide?
 - A hidden layer 'hides' its desired output.
 - Neurons in the hidden layer cannot be observed through the input/output behaviour of the network.
 - There is no obvious way to know what the desired output of the hidden layer should be.

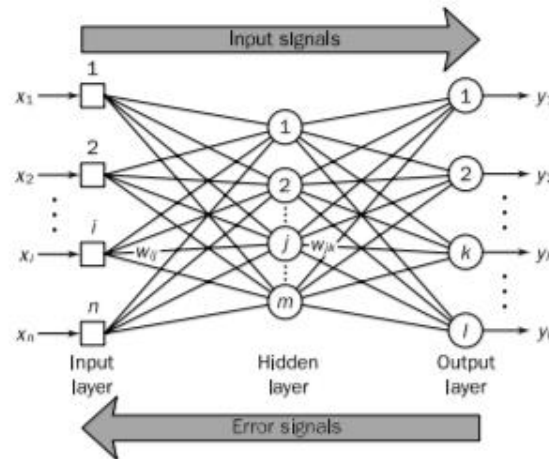
Feedforward neural network

- A **feedforward neural network** is an artificial neural network wherein connections between the units do *not* form a cycle.
- Data enters at the inputs and passes through the network, layer by layer, until it arrives at the outputs.
- During normal operation, that is when it acts as a classifier, there is no feedback between layers.
- This is why they are called *feedforward* neural networks.



Back-propagation Learning

- To derive the back-propagation learning law, let us consider the three-layer network shown below.
- Input signals, $x_1; x_2; \dots; x_n$, are propagated through the network from left to right, and error signals, $e_1; e_2; \dots; e_l$, from right to left.



Three-layer back-propagation neural network

Back-propagation Learning

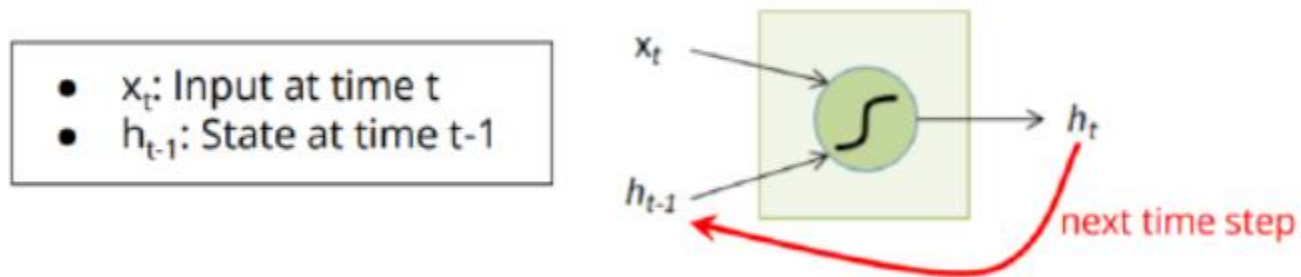
- To propagate error signals, we start at the output layer and work backward to the hidden layer.
- Error is estimated by subtracting desired and actual outputs.
- Weights are updated.

Recurrent Neural Networks

- A recurrent neural network (RNN) is a class of artificial neural network.
- A standard feed-forward neural network computes its output values in a sequential input-process-output manner.
- Unlike feedforward neural networks, RNNs can use their internal state (memory) to process sequences of inputs.
- RNNs are very powerful, because they combine two properties:
 - Distributed hidden state that allows them to store a lot of information about the past efficiently.
 - Non-linear dynamics that allows them to update their hidden state in complicated ways.
- With enough neurons and time, RNNs can compute anything that can be computed by your computer.

Cont.

The recurrent neuron is shown below:



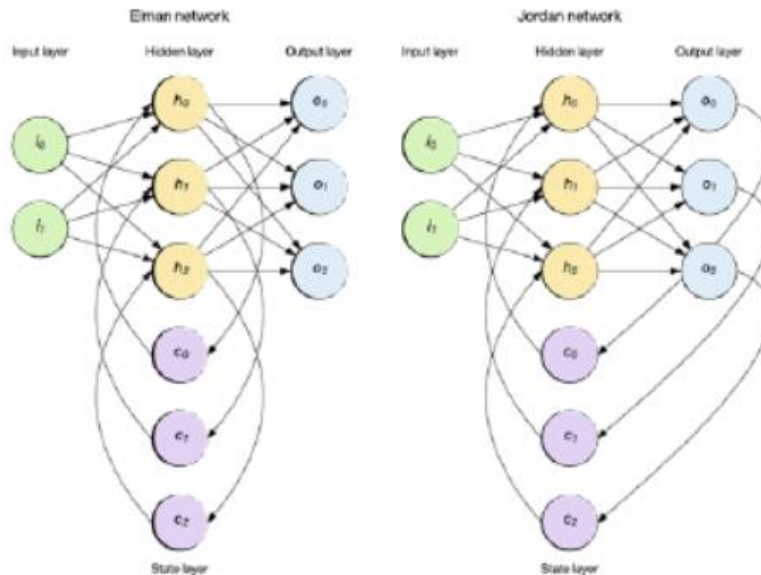
$$h_t = f(W_h h_{t-1} + W_x x_t)$$

Types of RNNs

- Elman Network
- Jordan Network
- Hopfield
- Bidirectional associative memory
- Independent RNN (IndRNN)
- Second order RNNs
- Long short-term memory
- Recurrent multilayer perceptron network
- And many others

Cont.

- Applications of **time-series data** require a new type of topology that can consider the **history** of the input. This is where RNNs can be applicable and very useful.
- Two popular recurrent network approaches are shown below:



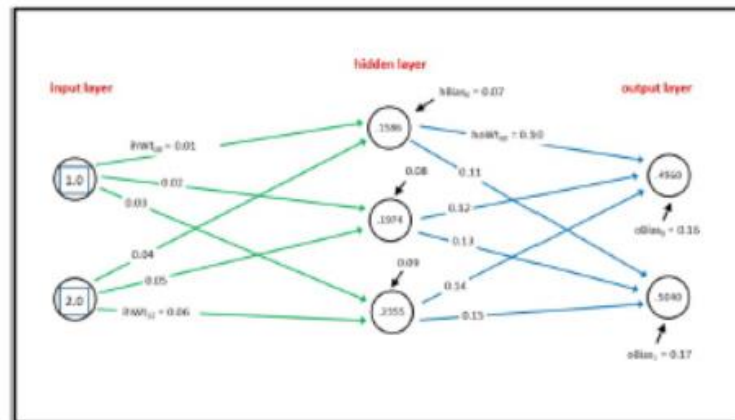
In the **Elman RNN**, the hidden layer feeds a state layer of context nodes that retain memory of past inputs. As shown in the following figure, a single set of context nodes exists that maintains memory of the prior hidden layer result.

In **Jordan RNN**, it stores the output layer into the state layer instead of maintaining history of the hidden layer.

- The state layer influences the next stage of input and therefore can be applied to time-varying patterns of data.

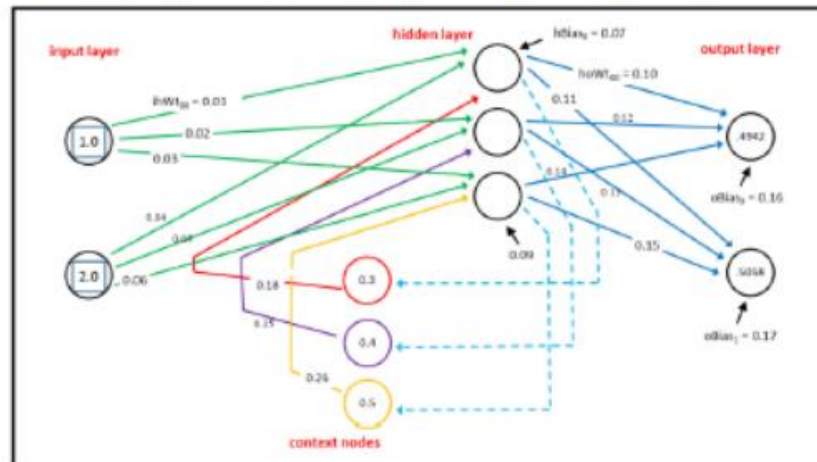
Elman RNN

- **Figure** shows a standard feed-forward neural network with two input nodes, three hidden nodes and two output nodes.
- The network has a total of 17 weights and biases.
- There are $2 * 3 = 6$ input-to-hidden weights with values 0.01 through 0.06.
- There are 3 hidden node biases with values 0.07, 0.08 and 0.09.
- There are $3 * 2 = 6$ hidden-to-output weights with values 0.10 through 0.15.
- And there are 2 output node biases with values 0.16 and 0.17.
- The neural network has input values of 1.0 and 2.0.



Cont.

- An obvious difference between a regular neural network and a recurrent neural network is that a recurrent network has more connections due to an additional set of nodes that are usually called **context nodes** or **context units**.
- There is a one-to-one correspondence between the hidden nodes and the context nodes. Because the network in the diagram has three hidden nodes, there are three context nodes.
- There is a feedback loop from each hidden node to its corresponding context node. In **Figure** these feedback loops are indicated by the blue dashed arrows.
- Each context node is forward-connected to corresponding hidden node with an associated weight. These weights are indicated by the red, purple and orange arrows.



Cont.

- In high-level pseudo code, the output values for a recurrent neural network are calculated like so:
 - *Compute preliminary hidden node sums using input values and context node values*
 - *Apply activation function to hidden node sums to get hidden node values*
 - *Compute preliminary output node sums using hidden node values*
 - *Apply activation function to output node sums to get final output node values*
 - *Copy hidden node values to associated context nodes for use with next input*

Other Neural Networks

- The Hopfield Neural Networks
- Bidirectional associative memory
- Self-organising Neural Networks

Generate a song using AI 😊

<https://aimusic.fm/community/>

References

- Flach, P. (2012). Machine Learning: The Art and Science of Algorithms that Make Sense of Data. Cambridge University Press.